



Neural network scene rendering

A team from Google's DeepMind have created an artificial intelligence that is exceptional at spatial reasoning. <https://dgit.in/Aug18-50-1>



IBM watermark for AI models

New patent-pending process allows deep learning models to be watermarked to protect against IP theft. <https://dgit.in/Aug18-50-2>

*CODING MATTERS



*MONGODB 4 Released

The popular document database had some support for ACID transactions at the document level in earlier versions, but until now not for multi-document transactions..

<https://dgit.in/MongoDB4>



*Bing adds Intelligent Code Search

There's a new search option in Bing called Code Sample Answer that you can use to find code that matches what you ask for in a question and return it as a code sample.

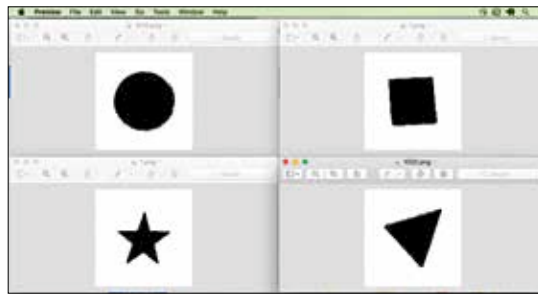
<https://dgit.in/BingCodeSearch>



*Apache Phoenix now HBase 2.0 compatible

Apache Phoenix 5.0 has been released. Phoenix adds support for SQL-based OLTP and operational analytics for Apache Hadoop using Apache HBase as its backing store.

<https://dgit.in/PhoenixHbase>



Reading inputs

We have used 600 images each of star, circle, square and triangle shapes from a Kaggle dataset but you could use any n image folders on your computer which contain different kinds of objects. Typically, we divide our input data into 3 parts:

1. Training data: we shall use 80% i.e. ~500 images for training.
2. Validation data: 20% images will be used for validation. These images are taken out to calculate accuracy independently during the training.
3. Test set: separate independent data for testing which has around 200 images. Sometimes due to a phenomenon called overfitting, after training, neural networks start working very well on the training data but they fail to work well for other images. For example, if you are training a classifier between shapes and you get training data from someone who takes all images with white backgrounds. It's possible that your network works very well on this data-set, but if you try to run it on an image with a yellow/red background, it will mostly fail.

The initial code needed to set this up can be found at <https://dgit.in/TF1>.

The Initial value of these weights can be taken anything but it works better if you take normal distributions. Let's now create functions to initialize weights quickly just by specifying the shape, as shown below.

```
def create_weights(shape):
    return tf.Variable(tf.
truncated_normal(shape, std-
dev=0.05))

def create_biases(size):
    return tf.Variable(tf.
constant(0.05, shape=[size]))
```

CREATING NETWORK LAYERS

Building convolution layer in TensorFlow

The `nn.conv2d` function can be used to create a convolutional layer. It takes the following inputs:

- input= the output from the previous layer.

Typically, in the first convolutional layer, you pass n images of size width*height*num_channels(3 for RGB), then this has the size `[n width height num_channels]`

- filter= trainable variables defining the parameters. We start with a random normal distribution and learn these weights. Filter will have following shape: `[filter_size filter_size num_input_channels num_filters]`
- strides= defines how much you move your filter for convolution.
- padding= SAME means we shall 0 pad the input such a way that output x,y dimensions are same as that of input.

Next, we use max-pooling using `nn.max_pool` function that's got a very similar signature as `conv2d`.

Finally, we use a RELU as our activation function which simply takes the output of `max_pool` and applies RELU using `nn.relu`. All these operations are done in a single convolution layer. A function for a complete convolutional layer is shown at <https://dgit.in/TF3>.

Flattening layer

The output of a convolutional layer is a multi-dimensional Tensor. To convert this to a one dimensional tensor we use flattening layer. We simply use the reshape operation to create a single dimensional tensor as defined at <https://dgit.in/TF4>.

Fully connected layer

Now, let's create a fully connected layer. Just like any other layer, we declare weights and biases as normal distributions. In a fully connected layer, we take the inputs and do the